

Paralelização de Aplicação com a Biblioteca MPI

Artur Marcon, Giovana Raupp e Vitoria Gonzalez

1 Introdução

Este trabalho implementa uma versão paralela do problema de geração de uma animação de *zoom* (dive) no conjunto de Julia, utilizando MPI no modelo coordenador/trabalhador. O saco contém N frames com fator de zoom progressivo $0,94^i$ e $\text{MAX_ITER} = 2000$ iterações por pixel.

2 Metodologia

Modelagem paralela. A *unidade de trabalho* é um frame de 1920×1080 (~ 6 MB de buffer RGB), despachado dinamicamente quando algum worker o solicita. O design respondeu a três questões: o custo é fortemente heterogêneo (zoom profundo atinge MAX_ITER com mais frequência, então divisão estática causaria grande desbalanceamento. Usamos **fila dinâmica com iniciativa do worker**); manter os 300 frames como matriz residente custaria $\sim 1,86$ GB no rank 0, então a ordem é preservada implicitamente pelo nome do arquivo (`frame_NNNN.ppm`), gravado quando cada resultado chega; o canal worker $\rightarrow$ mestre carrega mensagens de tamanhos distintos (~ 4 B vs ~ 6 MB), exigindo `MPI_Probe` (linha 141) antes do `MPI_Recv` apropriado e tags de resultado deslocadas (`TAG_RESULT_BASE + frame_id`, linha 14) para evitar colisão com `TAG_REQUEST = 0`.

Protocolo de comunicação. A iniciativa é dos trabalhadores: cada unidade é precedida por `TAG_REQUEST` explícito (linha 216). O mestre é puramente reativo e responde com `TAG_WORK` ou `TAG_DIE` quando o saco esvazia (linhas 152–156). Ao receber um resultado apenas grava e segue. A próxima dispatch só ocorre quando o worker pedir de novo. `MPI_Barrier` (linha 130) sincroniza os ranks antes de `MPI_Wtime`. Estatísticas por trabalhador são coletadas via `MPI_Gather` (linha 179) em canal isolado do dispatch, para que o mestre obtenha o tempo de cálculo de cada worker medido localmente (em volta de `compute_julia_frame`), informação necessária ao desbalanceamento e não inferível pela chegada dos resultados.

Ambiente experimental. Cluster *atlantica* (LAD/PU-CRS), 2 nós exclusivos alocados via `salloc -exclusive -N 2` dentro da mesma sessão (Ubuntu 20.04, gcc 9.4.0, OpenMPI 4.1.1; cada nó com 8 cores físicos / 16 threads HT). Tanto o baseline sequencial quanto a versão paralela executam em nós de cálculo da mesma alocação (o sequencial via `srun -n 1 -N 1`, garantindo hardware idêntico). Ambos compilados com `-O3 -Wall`. Cada configuração executada **3 vezes**; reporta-se a mediana. I/O de PPM desligado durante as medições para isolar custo de cálculo.

Métricas. Sendo W o número de *trabalhadores* ($W = np - 1$) e N a quantidade de frames do experimento:

$$S_f(W) = \frac{T_{\text{seq}}(60)}{T_{\text{par}}(W, 60)}, \quad E_f(W) = \frac{S_f(W)}{W}$$

$$S_w(W) = \frac{T_{\text{seq}}(N)}{T_{\text{par}}(W, N)}, \quad E_w(W) = \frac{S_w(W)}{W}$$

Para a escalabilidade fraca, fixa-se 10 frames por trabalhador ($N = 10W$). T_{seq} é medido para cada N no mesmo nó de cálculo da configuração paralela. O denominador da eficiência usa W (*contando apenas o trabalhador*). O *desbalanceamento* é $(\max - \min) / \max$ entre os tempos de cálculo dos trabalhadores.

3 Resultados

A Tabela 1 apresenta a escalabilidade forte (60 frames) e a Tabela 2 a fraca. Os gráficos correspondentes estão na Figura 1.

Escalabilidade forte. O speedup cresce de 0,99 ($W=1$, esperado) até $20,3 \times$ ($W=31$). A eficiência mantém-se acima de 88% até $W=15$ (0,88 em $W=15$, 0,92 em $W=7$, 0,98 em $W=3$) e cai para 66% em $W=31$. Os fatores limitantes observados foram: (i) o *rank 0* é serializado, recebendo um buffer RGB de ~ 6 MB por frame, processando o resultado, e respondendo ao próximo `TAG_REQUEST` com o próximo *frame id*; (ii) com apenas 60 frames distribuídos entre 31 trabalhadores, a granularidade torna-se grosseira: em média 1,9 frames por worker, ou seja, parte processa 2 frames e o restante apenas 1, gerando o salto de desbalanceamento de 15% ($W=15$) para 57% ($W=31$).

Escalabilidade fraca. Com T_{seq} medido para cada workload, a eficiência cai suavemente de 0,99 ($W=1$) para 0,75 ($W=31$, $N=310$), com speedup absoluto de $23,3 \times$. Esse resultado supera a escalabilidade forte ($E=0,66$ em $W=31$) porque a granularidade cresce com W (~ 10 frames/worker), amortizando melhor o custo de comunicação. A métrica ingênua $T_{\text{par}}(W=1)/T_{\text{par}}(W)$ daria 0,24 em $W=31$, enganosa por penalizar o paralelo pelos frames profundos serem mais caros que o baseline; $T_{\text{seq}}(N)/T_{\text{par}}(W, N)$ é honesta porque ambos termos enfrentam a mesma carga.

Balanceamento de carga. O modelo *coordenador/trabalhador*, com iniciativa dos trabalhadores, entrega balanceamento dinâmico naturalmente: workers que terminam antes pedem novo trabalho. O desbalanceamento medido cresce com W (2% em $W=3$, 13% em $W=7$, 15% em $W=15$, 57% em $W=31$), mas o salto em $W=31$ é *estrutural* (poucos frames por worker) e não um problema do algoritmo. Aumentar N recuperaria o balanceamento.

4 Conclusão

A versão MPI mestre/trabalhador atinge speedup forte de $20,3 \times$ ($E = 0,66$) e fraca de $23,3 \times$ ($E = 0,75$) com 31 trabalhadores, validando o modelo para carga heterogênea. A escalabilidade fraca supera a forte porque N cresce com W , amortizando o custo fixo de comunicação. As principais limitações observadas são: (a) serialização do rank 0 (recebe ~ 6 MB por frame), e (b) granularidade insuficiente em $W=31$ na escalabilidade forte (60 frames / 31 workers $\approx 1,9$). Trabalhos futuros incluem escrita de PPM delegada ao próprio trabalhador (reduz tráfego) e sobreposição de comunicação/computação com `MPI_Isend/MPI_Irecv`.

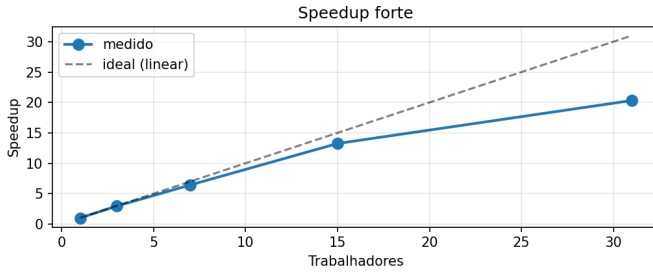
Tabelas e Gráficos

Tabela 1: Escalabilidade forte (60 frames, mediana de 3 reps). $T_{seq} = 163,91$ s.

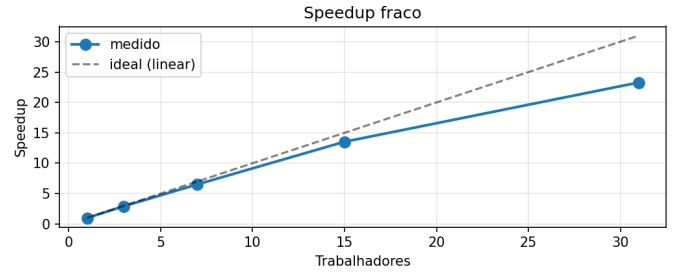
np	W	T (s)	S	E	Desbal. (%)
2	1	164,89	0,994	0,994	0,00
4	3	55,59	2,948	0,983	2,23
8	7	25,50	6,428	0,918	12,88
16	15	12,38	13,243	0,883	15,30
32	31	8,07	20,320	0,655	56,88

Tabela 2: Escalabilidade fraca ($N = 10W$, mediana de 3 reps p / T_{par} , 1 medição p / T_{seq}).

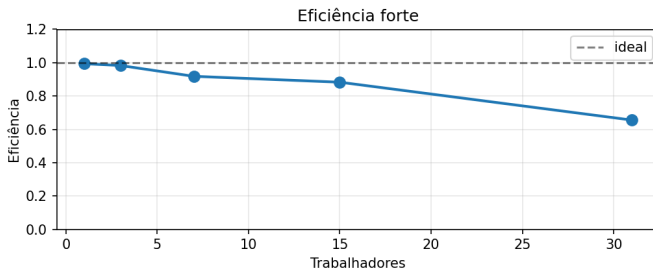
np	W	N	T_{seq} (s)	T_{par} (s)	S	E	Desb. (%)
2	1	10	17,78	17,95	0,991	0,991	0,00
4	3	30	70,33	24,08	2,920	0,973	3,99
8	7	70	198,95	30,50	6,523	0,932	10,15
16	15	150	556,20	41,10	13,532	0,902	10,11
32	31	310	1719,34	73,84	23,285	0,751	15,62



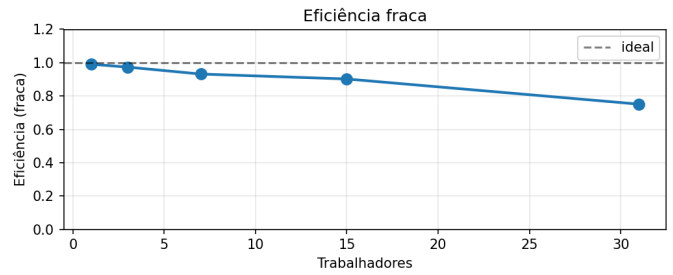
(a) Speedup forte



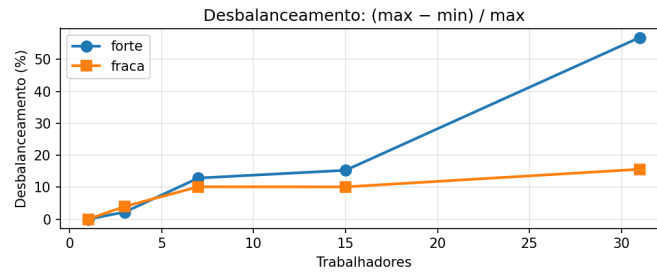
(b) Speedup fraco



(c) Eficiência forte



(d) Eficiência fraca



(e) Desbalanceamento

Figura 1: Curvas de escalabilidade e balanceamento de carga.

Código-fonte

julia_dive.c - versão paralela MPI (mestre/trabalhador)

```
1  #include <mpi.h>
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <math.h>
5
6  #define WIDTH 1920
7  #define HEIGHT 1080
8  #define MAX_ITER 2000
9  #define DEFAULT_TOTAL_FRAMES 300
10
11 #define TAG_REQUEST 0
12 #define TAG_WORK 1
13 #define TAG_DIE 2
14 #define TAG_RESULT_BASE 3
15
16 const double C_REAL = -0.7;
17 const double C_IMAG = 0.27015;
18
19 const int COLOR1_R = 10;  const int COLOR1_G = 15;  const int COLOR1_B = 45;
20 const int COLOR2_R = 0;   const int COLOR2_G = 225; const int COLOR2_B = 235;
21 const int COLOR3_R = 140; const int COLOR3_G = 25;  const int COLOR3_B = 230;
22
23 void compute_julia_frame(int frame_id, double center_real, double center_imag, unsigned char *buffer) {
24     double zoom = pow(0.94, frame_id);
25
26     double base_width = 3.0;
27     double current_width = base_width * zoom;
28     double current_height = current_width * ((double)HEIGHT / (double)WIDTH);
29
30     for (int y = 0; y < HEIGHT; y++) {
31         for (int x = 0; x < WIDTH; x++) {
32             double z_real = center_real + (x - WIDTH / 2.0) * (current_width / WIDTH);
33             double z_imag = center_imag + (y - HEIGHT / 2.0) * (current_height / HEIGHT);
34
35             int iter = 0;
36             double z_real_sq = z_real * z_real;
37             double z_imag_sq = z_imag * z_imag;
38
39             while (z_real_sq + z_imag_sq <= 4.0 && iter < MAX_ITER) {
40                 double temp = z_real_sq - z_imag_sq + C_REAL;
41                 z_imag = 2.0 * z_real * z_imag + C_IMAG;
42                 z_real = temp;
43
44                 z_real_sq = z_real * z_real;
45                 z_imag_sq = z_imag * z_imag;
46                 iter++;
47             }
48
49             size_t pixel_idx = (size_t)(y * WIDTH + x) * 3;
50
51             if (iter == MAX_ITER) {
52                 buffer[pixel_idx] = 0;
53                 buffer[pixel_idx + 1] = 0;
54                 buffer[pixel_idx + 2] = 0;
55             } else {
56                 double mu = (double)iter / MAX_ITER;
57
58                 if (mu < 0.5) {
59                     double t = mu * 2.0;
60                     buffer[pixel_idx] = (unsigned char)((1.0 - t) * COLOR1_R + t * COLOR2_R);
61                     buffer[pixel_idx + 1] = (unsigned char)((1.0 - t) * COLOR1_G + t * COLOR2_G);
62                     buffer[pixel_idx + 2] = (unsigned char)((1.0 - t) * COLOR1_B + t * COLOR2_B);
63                 } else {
64                     double t = (mu - 0.5) * 2.0;
65                     buffer[pixel_idx] = (unsigned char)((1.0 - t) * COLOR2_R + t * COLOR3_R);
66                     buffer[pixel_idx + 1] = (unsigned char)((1.0 - t) * COLOR2_G + t * COLOR3_G);
67                     buffer[pixel_idx + 2] = (unsigned char)((1.0 - t) * COLOR2_B + t * COLOR3_B);
68                 }
69             }
70         }
71     }
72 }
73
74 void save_ppm(int frame_id, unsigned char *buffer) {
75     char filename[64];
76     sprintf(filename, "frame_%04d.ppm", frame_id);
77
78     FILE *fp = fopen(filename, "wb");
79     if (!fp) {
80         fprintf(stderr, "Error opening file %s for writing.\n", filename);
81         return;
82     }
83
84     fprintf(fp, "P6\n%d %d\n255\n", WIDTH, HEIGHT);
85     fwrite(buffer, sizeof(unsigned char), (size_t)WIDTH * HEIGHT * 3, fp);
86     fclose(fp);
87 }
```

```

90
91 int main(int argc, char *argv[]) {
92     int rank, size;
93
94     MPI_Init(&argc, &argv);
95     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
96     MPI_Comm_size(MPI_COMM_WORLD, &size);
97
98     double target_real = -0.1;
99     double target_imag = -0.1;
100     int total_frames = DEFAULT_TOTAL_FRAMES;
101     int save_files = 1;
102
103     if (argc >= 3) {
104         target_real = atof(argv[1]);
105         target_imag = atof(argv[2]);
106     }
107     if (argc >= 4) {
108         total_frames = atoi(argv[3]);
109     }
110     if (argc >= 5) {
111         save_files = atoi(argv[4]);
112     }
113
114     if (size < 2) {
115         if (rank == 0) {
116             fprintf(stderr, "Error: This Master-Worker model requires at least 2 MPI processes.\n");
117             fprintf(stderr, "Usage: %s [target_real target_imag [total_frames [save_files]]]\n", argv[0]);
118         }
119         MPI_Finalize();
120         return 1;
121     }
122
123     size_t frame_buffer_size = (size_t)WIDTH * HEIGHT * 3 * sizeof(unsigned char);
124
125     if (rank == 0) {
126
127         printf("[Master] Rendering %d frames into (%f, %f), workers=%d, save_files=%d\n",
128             total_frames, target_real, target_imag, size - 1, save_files);
129
130         MPI_Barrier(MPI_COMM_WORLD);
131         double start_time = MPI_Wtime();
132         double io_time = 0.0;
133
134         int next_frame = 0;
135         int frames_saved = 0;
136         int active_workers = size - 1;
137
138         unsigned char *recv_buffer = (unsigned char *)malloc(frame_buffer_size);
139         while (frames_saved < total_frames || active_workers > 0) {
140             MPI_Status status;
141             MPI_Probe(MPI_ANY_SOURCE, MPI_ANY_TAG, MPI_COMM_WORLD, &status);
142
143             int worker_id = status.MPI_SOURCE;
144             int tag = status.MPI_TAG;
145
146             if (tag == TAG_REQUEST) {
147                 int dummy;
148                 MPI_Recv(&dummy, 1, MPI_INT, worker_id, TAG_REQUEST,
149                     MPI_COMM_WORLD, MPI_STATUS_IGNORE);
150
151                 if (next_frame < total_frames) {
152                     MPI_Send(&next_frame, 1, MPI_INT, worker_id, TAG_WORK, MPI_COMM_WORLD);
153                     next_frame++;
154                 } else {
155                     int term = -1;
156                     MPI_Send(&term, 1, MPI_INT, worker_id, TAG_DIE, MPI_COMM_WORLD);
157                     active_workers--;
158                 }
159             } else {
160
161                 int completed_frame_id = tag - TAG_RESULT_BASE;
162                 MPI_Recv(recv_buffer, (int)frame_buffer_size, MPI_BYTE,
163                     worker_id, tag, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
164                 if (save_files) {
165                     double io0 = MPI_Wtime();
166                     save_ppm(completed_frame_id, recv_buffer);
167                     io_time += MPI_Wtime() - io0;
168                 }
169                 frames_saved++;
170             }
171         }
172
173         free(recv_buffer);
174         double end_time = MPI_Wtime();
175         double total_time = end_time - start_time;
176
177         double local_stats[2] = {0.0, 0.0};
178         double *all_stats = (double *)malloc(2 * size * sizeof(double));
179         MPI_Gather(local_stats, 2, MPI_DOUBLE, all_stats, 2, MPI_DOUBLE, 0, MPI_COMM_WORLD);
180
181         printf("[Master] total=%.4fs io=%.4fs comm+wait=%.4fs\n",
182             total_time, io_time, total_time - io_time);
183         printf("[Master] worker_id,frames_processed,compute_time_s\n");

```

```

184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
}

int total_frames_check = 0;
double max_compute = 0.0, min_compute = 1e18, sum_compute = 0.0;
for (int worker = 1; worker < size; worker++) {
    int wframes = (int)all_stats[2 * worker];
    double wtime = all_stats[2 * worker + 1];
    total_frames_check += wframes;
    if (wtime > max_compute) max_compute = wtime;
    if (wtime < min_compute) min_compute = wtime;
    sum_compute += wtime;
    printf("[Master] %d,%d,%.4f\n", worker, wframes, wtime);
}
double avg_compute = sum_compute / (size - 1);
double imbalance = (max_compute - min_compute) / (max_compute > 0 ? max_compute : 1.0);
printf("[Master] frames_total=%d avg_compute=%.4fs max=%.4f min=%.4f imbalance=%.2f%%\n",
    total_frames_check, avg_compute, max_compute, min_compute, imbalance * 100.0);

printf("CSV,par,%d,%d,%d,%.6f,%.6f,%.6f,%.6f,%.4f\n",
    size, size - 1, total_frames,
    total_time, io_time, max_compute, min_compute, imbalance * 100.0);
free(all_stats);

} else {

    unsigned char *local_buffer = (unsigned char *)malloc(frame_buffer_size);
    int frames_processed = 0;
    double compute_time = 0.0;
    int request_payload = rank;

    MPI_Barrier(MPI_COMM_WORLD);

    while (1) {
        MPI_Send(&request_payload, 1, MPI_INT, 0, TAG_REQUEST, MPI_COMM_WORLD);

        int frame_to_build;
        MPI_Status status;
        MPI_Recv(&frame_to_build, 1, MPI_INT, 0, MPI_ANY_TAG, MPI_COMM_WORLD, &status);

        if (status.MPI_TAG == TAG_DIE) {
            break;
        }

        double c0 = MPI_Wtime();
        compute_julia_frame(frame_to_build, target_real, target_imag, local_buffer);
        compute_time += MPI_Wtime() - c0;
        frames_processed++;

        MPI_Send(local_buffer, (int)frame_buffer_size, MPI_BYTE, 0,
            TAG_RESULT_BASE + frame_to_build, MPI_COMM_WORLD);
    }

    double local_stats[2] = { (double)frames_processed, compute_time };
    MPI_Gather(local_stats, 2, MPI_DOUBLE, NULL, 0, MPI_DOUBLE, 0, MPI_COMM_WORLD);

    free(local_buffer);
}

MPI_Finalize();
return 0;
}

```

julia_dive_seq.c - versão sequencial (baseline)

```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <time.h>

#define WIDTH 1920
#define HEIGHT 1080
#define MAX_ITER 2000
#define DEFAULT_TOTAL_FRAMES 300

const double C_REAL = -0.7;
const double C_IMAG = 0.27015;

const int COLOR1_R = 10;    const int COLOR1_G = 15;    const int COLOR1_B = 45;
const int COLOR2_R = 0;     const int COLOR2_G = 225;   const int COLOR2_B = 235;
const int COLOR3_R = 140;   const int COLOR3_G = 25;    const int COLOR3_B = 230;

void compute_julia_frame(int frame_id, double center_real, double center_imag, unsigned char *buffer) {
    double zoom = pow(0.94, frame_id);

    double base_width = 3.0;
    double current_width = base_width * zoom;
    double current_height = current_width * ((double)HEIGHT / (double)WIDTH);

    for (int y = 0; y < HEIGHT; y++) {
        for (int x = 0; x < WIDTH; x++) {
            double z_real = center_real + (x - WIDTH / 2.0) * (current_width / WIDTH);
            double z_imag = center_imag + (y - HEIGHT / 2.0) * (current_height / HEIGHT);

```

```

29
30     int iter = 0;
31     double z_real_sq = z_real * z_real;
32     double z_imag_sq = z_imag * z_imag;
33
34     while (z_real_sq + z_imag_sq <= 4.0 && iter < MAX_ITER) {
35         double temp = z_real_sq - z_imag_sq + C_REAL;
36         z_imag = 2.0 * z_real * z_imag + C_IMAG;
37         z_real = temp;
38
39         z_real_sq = z_real * z_real;
40         z_imag_sq = z_imag * z_imag;
41         iter++;
42     }
43
44     size_t pixel_idx = (size_t)(y * WIDTH + x) * 3;
45
46     if (iter == MAX_ITER) {
47         buffer[pixel_idx] = 0;
48         buffer[pixel_idx + 1] = 0;
49         buffer[pixel_idx + 2] = 0;
50     } else {
51         double mu = (double)iter / MAX_ITER;
52
53         if (mu < 0.5) {
54             double t = mu * 2.0;
55             buffer[pixel_idx] = (unsigned char)((1.0 - t) * COLOR1_R + t * COLOR2_R);
56             buffer[pixel_idx + 1] = (unsigned char)((1.0 - t) * COLOR1_G + t * COLOR2_G);
57             buffer[pixel_idx + 2] = (unsigned char)((1.0 - t) * COLOR1_B + t * COLOR2_B);
58         } else {
59             double t = (mu - 0.5) * 2.0;
60             buffer[pixel_idx] = (unsigned char)((1.0 - t) * COLOR2_R + t * COLOR3_R);
61             buffer[pixel_idx + 1] = (unsigned char)((1.0 - t) * COLOR2_G + t * COLOR3_G);
62             buffer[pixel_idx + 2] = (unsigned char)((1.0 - t) * COLOR2_B + t * COLOR3_B);
63         }
64     }
65 }
66 }
67 }
68
69 void save_ppm(int frame_id, unsigned char *buffer) {
70     char filename[64];
71     sprintf(filename, "frame_%04d.ppm", frame_id);
72
73     FILE *fp = fopen(filename, "wb");
74     if (!fp) {
75         fprintf(stderr, "Error opening file %s for writing.\n", filename);
76         return;
77     }
78
79     fprintf(fp, "P6\n%d %d\n255\n", WIDTH, HEIGHT);
80     fwrite(buffer, sizeof(unsigned char), (size_t)WIDTH * HEIGHT * 3, fp);
81     fclose(fp);
82 }
83
84 static double now_seconds(void) {
85     struct timespec ts;
86     clock_gettime(CLOCK_MONOTONIC, &ts);
87     return (double)ts.tv_sec + (double)ts.tv_nsec / 1e9;
88 }
89
90 int main(int argc, char *argv[]) {
91     double target_real = -0.1;
92     double target_imag = -0.1;
93     int total_frames = DEFAULT_TOTAL_FRAMES;
94     int save_files = 1;
95
96     if (argc >= 3) {
97         target_real = atof(argv[1]);
98         target_imag = atof(argv[2]);
99     }
100     if (argc >= 4) {
101         total_frames = atoi(argv[3]);
102     }
103     if (argc >= 5) {
104         save_files = atoi(argv[4]);
105     }
106
107     size_t frame_buffer_size = (size_t)WIDTH * HEIGHT * 3 * sizeof(unsigned char);
108     unsigned char *buffer = (unsigned char *)malloc(frame_buffer_size);
109     if (!buffer) {
110         fprintf(stderr, "Error: failed to allocate frame buffer.\n");
111         return 1;
112     }
113
114     printf("[Seq] Rendering %d frames into (%f, %f), save_files=%d\n",
115           total_frames, target_real, target_imag, save_files);
116
117     double t_compute = 0.0;
118     double t_io = 0.0;
119     double t_start = now_seconds();
120
121     for (int frame_id = 0; frame_id < total_frames; frame_id++) {
122         double tc0 = now_seconds();

```

```

123     compute_julia_frame(frame_id, target_real, target_imag, buffer);
124     double tc1 = now_seconds();
125     t_compute += (tc1 - tc0);
126
127     if (save_files) {
128         double ti0 = now_seconds();
129         save_ppm(frame_id, buffer);
130         double ti1 = now_seconds();
131         t_io += (ti1 - ti0);
132     }
133 }
134
135 double t_end = now_seconds();
136 double t_total = t_end - t_start;
137
138 printf("[Seq] total=%.4fs  compute=%.4fs  io=%.4fs\n", t_total, t_compute, t_io);
139
140 printf("CSV,seq,1,0,%d,%.6f,%.6f,%.6f,%.6f,0.0000\n",
141        total_frames, t_total, t_io, t_compute, t_compute);
142
143 free(buffer);
144 return 0;
145 }

```

run_experiments.sh - script de experimentos

```

1  #!/usr/bin/env bash
2
3  set -euo pipefail
4
5  MODE="${1:-local}"
6  REPS="${REPS:-3}"
7
8  case "$MODE" in
9      local)
10         NP_LIST=(2 4 8)
11         FRAMES_STRONG="${FRAMES_STRONG:-60}"
12         FRAMES_PER_WORKER="${FRAMES_PER_WORKER:-10}"
13         LAUNCHER=(mpirun -np)
14         SEQ_PREFIX=()
15         ;;
16      lad)
17         NP_LIST=(2 4 8 16 32)
18         FRAMES_STRONG="${FRAMES_STRONG:-300}"
19         FRAMES_PER_WORKER="${FRAMES_PER_WORKER:-20}"
20         LAUNCHER=(srun -N 2 -n)
21         SEQ_PREFIX=(srun -n 1 -N 1)  # -N 1 silences "1 proc on 2 nodes" warning
22         ;;
23      *)
24         echo "usage: $0 [local|lad]" >&2
25         exit 1
26         ;;
27  esac
28
29  OUT="results_${MODE}.csv"
30  echo "experiment,rep,kind,np,workers,total_frames,total_s,io_s,compute_max_s,compute_min_s,imbalance_pct" > "$OUT"
31
32  run_seq() {
33      local exp="$1" rep="$2" frames="$3"
34      echo "[run] $exp rep=$rep seq frames=$frames"
35      local line
36      line=$(("${SEQ_PREFIX[@]}" ./julia_dive_seq -0.1 -0.1 "$frames" 0 | grep '^CSV,' | sed 's/^CSV,/'))
37      echo "$exp,$rep,$line" >> "$OUT"
38  }
39
40  run_par() {
41      local exp="$1" rep="$2" np="$3" frames="$4"
42      echo "[run] $exp rep=$rep np=$np frames=$frames"
43      local line
44      line=$(("${LAUNCHER[@]}" "$np" ./julia_dive -0.1 -0.1 "$frames" 0 | grep '^CSV,' | sed 's/^CSV,/'))
45      echo "$exp,$rep,$line" >> "$OUT"
46  }
47
48  echo "=== Strong scaling (frames=$FRAMES_STRONG) ==="
49  for rep in $(seq 1 "$REPS"); do
50      run_seq strong "$rep" "$FRAMES_STRONG"
51      for np in "${NP_LIST[@]}; do
52          run_par strong "$rep" "$np" "$FRAMES_STRONG"
53      done
54  done
55
56  echo "=== Weak scaling (~$FRAMES_PER_WORKER frames/worker) ==="
57  for rep in $(seq 1 "$REPS"); do
58      for np in "${NP_LIST[@]}; do
59          workers=$((np - 1))
60          frames=$((workers * FRAMES_PER_WORKER))
61          run_par weak "$rep" "$np" "$frames"
62      done
63  done
64
65  echo

```

```
66 echo "Done. Results in: $OUT"
```